RamBot Architecture Workflow Report

Current-state architecture summary derived from the repository structure, runtime manifests, service entrypoints, frontend code, and dependency files present in this workspace on 2026-04-17.

Monorepo: TurboRepo | Web: Next.js 14.2.4 | Mobile: Expo 54 / RN 0.81.5 | Backend: FastAPI 0.111.0 | AI: Groq / NVIDIA / Cerebras / Ollama

REPOSITORY

rambot-enterprise 1.0.0

NODE TOOLCHAIN

Node >= 20.0.0, npm >= 10.0.0, packageManager npm 11.4.2

PYTHON RUNTIMES

Python 3.12-slim for most services, Python 3.11-slim for document-service

DEPLOYMENT MODEL

Docker Compose locally, Kubernetes manifests included

Solution Shape

Client Layer

Two frontends exist: `apps/web` and `apps/mobile`.

- Web uses App Router pages for Steer, Skill, and Documents.
- Mobile uses Expo Router with tab navigation.

Domain Services

The backend is split into API Gateway, Steer, Skill, Auth, AI Orchestrator, Notification, and Document Service.

- Steer and Skill follow presentation, application, domain, infrastructure layers.
- Document Service is pipeline-oriented for async PDF analysis.

Platform Layer

Infrastructure centers on PostgreSQL, Redis, RabbitMQ, Docker Compose, and optional Kubernetes.

- Shared TS packages expose API client and types.
- AI providers are configurable across local and cloud runtimes.

Architecture Workflow

CLIENT

Web App

Next.js 14.2.4, React 18.2.0, Redux Toolkit, React Query, Sass, Mermaid, Axios.

CLIENT

Mobile App

Expo 54.0.33, React 19.1.0, React Native 0.81.5, Expo Router, React Navigation.

SHARED PACKAGES

API Contracts

`packages/shared-types` and `packages/api-client` centralize DTOs and JS access.

Web and mobile use the gateway for most domain APIs. The document UI also talks directly to document-service.

PORT 8000

API Gateway

FastAPI gateway with auth middleware, rate limiting, logging, gzip, CORS, and reverse proxy routes.

PORT 8001

Steer Service

Strategic goal management with async SQLAlchemy, Redis cache, RabbitMQ publishing, and AI scoring support.

PORT 8002

Skill Service

Skill catalog and lifecycle transitions backed by SQLAlchemy, Redis, RabbitMQ, and AI helper logic.

PORT 8003

Auth Service

Auth API scaffold exists, but login/register/refresh/me currently return 501 placeholders.

PORT 8004

AI Orchestrator

Dedicated FastAPI service for steer analysis and skill gap analysis via Ollama-backed agents.

PORTS 8005 / 8006

Notification + Document

Notification consumes events; Document handles upload, validation, background analysis, and retrieval APIs.

Persistence, caching, and messaging sit behind the services.

DATA

PostgreSQL 16-alpine

Compose provisions steer, skill, and auth databases. Document code currently uses in-memory storage helpers.

CACHE

Redis 7-alpine

Used for rate limiting and service-local caching with separate DB indexes per service.

MESSAGING

RabbitMQ 3.13-management-alpine

Event broker for publication/consumption. Some services fall back to in-memory publishers if unavailable.

AI RUNTIME

Cloud + Local LLM

Groq, NVIDIA NIM, Cerebras, and Ollama are supported depending on service and environment settings.

Observed implementation nuance: the web document feature uses `NEXT_PUBLIC_DOCUMENT_API_URL` and calls document-service directly for uploads and analysis polling, while other feature areas use `NEXT_PUBLIC_API_URL` through the gateway.

Primary Workflows

1 Steer Workflow

Clients send Steer requests to API Gateway, then the gateway proxies them to Steer service.

- Gateway applies logging, auth, rate limiting, compression, and CORS.
- Steer endpoints are fully wired with FastAPI dependency injection.
- Startup initializes PostgreSQL tables, Redis, RabbitMQ, and an LLM client.
- LLM fallback chain in Steer is Groq → NVIDIA NIM → Cerebras → mock.

2 Skill Workflow

Skill traffic also enters through API Gateway and is handled by the Skill service lifecycle workflows.

- Supported actions include create, list, search, submit-review, approve, deploy, metrics update, and deprecate.

- Skill mirrors the same infrastructure pattern as Steer: PostgreSQL, Redis, RabbitMQ, and multi-provider AI fallback.

3 Document Analysis Workflow

The document feature is the clearest end-to-end async workflow in the repo.

- User uploads a PDF from the web app directly to `document-service` on port 8006.
- Document Service validates file type, size, and empty input.
- Upload returns document metadata and starts a FastAPI background task.
- The background task extracts text and runs three AI chains in parallel: summary, entity extraction, and Mermaid workflow generation.
- Frontend polls `/api/v1/analyses/{id}` every 3 seconds until status becomes `done` or `failed`.

4 AI Orchestration Workflow

AI Orchestrator exposes separate endpoints for steer analysis and skill gap analysis.

- It creates specialized agents and sends context to Ollama-backed analysis routines.
- Current orchestrator settings default to `OLLAMA_MODEL=llama3.2`.
- Document-service AI does not use the orchestrator; it uses its own provider fallback logic.

5 Notifications and Events

Notification service is positioned as the event consumer layer.

- RabbitMQ is part of the compose stack and notification-service depends on it.
- Firebase push support is present through `firebase-admin==6.5.0`.
- Steer and Skill publish events or fall back to in-memory publishers if the broker is unavailable.

Technology Stack With Versions

LAYER	TECHNOLOGY	VERSION	OBSERVED USAGE
Monorepo	TurboRepo	`^2.0.0`	Root build/dev/test orchestration.
Node Toolchain	Node.js / npm	`>=20.0.0` / `>=10.0.0`	Root engines; package manager pinned to npm 11.4.2.
Web Framework	Next.js	`14.2.4`	Web client in `apps/web`.
Web UI	React / React DOM	`18.2.0`	Web rendering and App Router pages.
Web State	Redux Toolkit	`^2.2.5`	Feature stores for steer, skill, document, auth.
Web Data	TanStack React Query	`^5.45.0`	Available in web dependencies for server-state work.
Web Styling	Sass / Tailwind CSS	`^1.77.0` / `3.4.4`	Mixed styling approach in the web app.
Web Visualization	Mermaid / Recharts / XYFlow	`11.14.0` / `^2.12.7` / `^12.3.6`	Diagramming and visual components.
HTTP Client	Axios	`^1.15.0`	Used by shared API client and some slices.
Mobile Framework	Expo	`~54.0.33`	Mobile app runtime.
Mobile UI	React / React Native	`19.1.0` / `0.81.5`	Mobile client runtime.
Mobile Routing	Expo Router	`~6.0.23`	File-based mobile navigation.
Mobile Navigation	React Navigation	`@react-navigation/native` `7.1.8`	Tabs and navigation primitives.

LAYER	TECHNOLOGY	VERSION	OBSERVED USAGE
Backend Framework	FastAPI	`0.111.0`	Used by gateway and all Python services.
ASGI Server	Uvicorn	`0.29.0`	Service runtime.
Python Validation	Pydantic / pydantic-settings	`2.7.0` / `2.2.1`	Config and request/response models.
ORM	SQLAlchemy asyncio / asyncpg / Alembic	`2.0.30` / `0.29.0` / `1.13.1`	Steer, Skill, and Auth persistence stack.
Cache	redis Python client / Redis image	`5.0.4` / `7-alpine`	Rate limiting and per-service cache.
Messaging	aio-pika / RabbitMQ	`9.4.1` / `3.13-management-alpine`	Async event publishing and broker runtime.
Database	PostgreSQL	`16-alpine`	Compose-managed relational storage.
Auth/JWT	python-jose / PyJWT / passlib[bcrypt]	`3.3.0` / `2.8.0` / `1.7.4`	JWT verification and password hashing dependencies.
Observability	Prometheus / OpenTelemetry / Sentry / structlog	`7.0.0` / `1.24.0` / `2.1.1` / `24.1.0`	Gateway observability stack.
Document Parsing	pypdf / jsonschema	`4.2.0` / `4.22.0`	PDF extraction and structured validation.
AI SDKs	OpenAI SDK / Groq SDK / Ollama Python	`1.35.0` / `0.11.0` / `0.3.3`	Provider integrations across services.
Push Notifications	firebase-admin	`6.5.0`	Notification provider integration.
Container Runtimes	Python base images	`3.12-slim` and `3.11-slim`	Document-service differs from the rest.

Service Inventory

API Gateway

`services/api-gateway`

Routes: auth, steer, skill, document, health.

Middleware: auth, rate limit, logging, gzip, CORS.

Steer Service

`services/steer-service`

Goal CRUD and lifecycle transitions.

Uses SQLAlchemy, Redis, RabbitMQ, AI fallback chain.

Skill Service

`services/skill-service`

Skill CRUD, review, deploy, metrics, deprecate.

Mirrors the Steer layered architecture.

Auth Service

`services/auth-service`

Dependency stack exists, but main REST actions are still stubs returning 501.

AI Orchestrator

`services/ai-orchestrator`

Dedicated service for steer and skill AI analysis.

Document Service

`services/document-service`

Upload + async pipeline + poll-based retrieval.

Notification Service

`services/notification-service`

RabbitMQ-driven notifications with FCM support.

Infrastructure

`infrastructure/docker` and `infrastructure/kubernetes`

Compose for local orchestration, K8s manifests for deployment.

Implementation Notes and Gaps

What Is Strong

- Clear service decomposition by domain.
- Steer and Skill use layered architecture consistently.
- Frontend is split by features and aligned to domain areas.
- Document-service has a concrete async processing pattern.
- Multi-provider AI fallback is already implemented.

What Looks Transitional

- Auth service endpoints are not yet wired to real use cases.
- Document feature bypasses the gateway, so ingress is not fully uniform.
- Document-service currently uses in-memory storage helpers in code.
- Python base image versions are not fully standardized.

- Some older architecture markdown is broader than the currently wired implementation.

This report reflects the current implementation in code, not only the intended target architecture. The repository contains both implemented flows and planned or stubbed capabilities.

Saved as `Rambot_Architecture_Workflow_Report.html`. Open it in a browser and print to PDF if you need a PDF copy.